CHAPTER

13

UART Projects

13.1 OVERVIEW

In this chapter we shall be developing projects using the serial communication module UART with the Nucleo-F411RE development board.

Serial communication is a simple means of sending data to long distances quickly and reliably. The most commonly used serial communication method is based on the RS232 standard. In this standard data is sent over a single line from a transmitting device to a receiving device in bit serial format at a prespecified speed, also known as the Baud rate, or the number of bits sent each second. Typical Baud rates are 4800, 9600, 19200, 38400, etc.

RS232 serial communication is a form of asynchronous data transmission where data is sent character by character. Each character is preceded with a Start bit, seven or eight data bits, an optional parity bit (check bit), and one or more stop bits. The most commonly used format is eight data bits, no parity bit, and one stop bit. The least significant data bit is transmitted first, followed by the other bits, and the most significant bit transmitted last.

Data in serial asynchronous communication can either be sent using the standard RS232 protocol or two devices can simply be connected together provided they satisfy the logic gate interface rules (e.g., TTL to TTL, or TTL to CMOS, etc.). In the RS232 protocol, a logic HIGH is defined to be at −12 V, and a logic 0 is at +12 V. Fig. 13.1 shows how character **A** (ASCII binary pattern 0010 0001) is transmitted over a serial line via the RS232 protocol. The line is normally idle at −12 V. The start bit is first sent by the line going from HIGH to LOW. Then eight data bits are sent starting from the least significant bit. Finally the stop bit is sent by raising the line from LOW to HIGH.

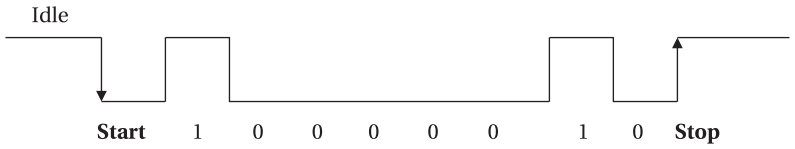

FIG. 13.1 Sending character "A" in serial format.

https://doi.org/10.1016/B978-0-08-102969-5.00013-6

In serial connection a minimum of three lines are used for communication: transmit (TX), receive (RX), and ground (GND). Two devices are used in serial communication: the transmitter, and the receiver. The devices are connected such that the TX of one device is connected to the RX of the other device, and its RX is connected to the TX. Some high-speed serial communication systems use additional control signals for synchronization, such as CTS, DTR, RTS, and so on. Some systems use software synchronization techniques where a special character (XOFF) is used to tell the sender to stop sending, and another character (XON) is used to tell the sender to restart transmission. In this chapter we will be using low-speed communication and therefore the basic pins presented in Table 13.1 will be used with no hardware or software synchronization.

Serial devices are connected to each other using two types of connectors: 9-way connector, or 25-way connector. Table 13.1 illustrates the TX, RX, and GND pins of each types of connectors. The connectors used in RS232 serial communication are shown in Fig. 13.2.

As described above, RS232 voltage levels are at ±12 V. On the other hand, microcontroller input-output ports operate at 0 to +3.3 V or 0 to +5 V voltage levels. It is therefore necessary to translate the voltage levels before a microcontroller can be connected to an RS232 compatible device. Thus, the output signal from the microcontroller has to be converted into ±12 V, and the input from an RS232 device must be converted into 0 to +5 V before it can be connected to a microcontroller. This voltage translation is normally done using special RS232 voltage converter chips. One such popular chip is the MAX232. In the UART project in this chapter

TABLE 13.1 Minimum Required Pins for Serial Communication

Pin	Function
9-PIN CONNECTOR	
2	Transmit (TX)
3	Receive (RX)
5	Ground (GND)
25-PIN CONNECTOR	
2	Transmit (TX)
3	Receive (RX)
7	Ground (GND)

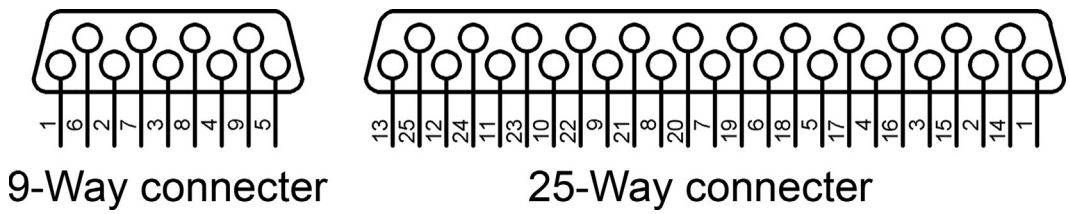

FIG. 13.2 RS232 connectors.

the RS232 protocol is not used and the two communicating devices are connected to each other directly.

Table 13.2 illustrates the functions supported by Mbed UART.

TABLE 13.2 Mbed UART Functions

Function	Description
baud	Set the serial communication baud rate
format	Set the communication format
getc	Read a character from serial port
putc	Write a character to serial port
printf	Write formatted string to serial port
scanf	Read formatted string from serial port
readable	Determine if a there a character available to read
writeable	Determine if there is space to write a character
attach	Attach an interrupt service routine function to serial port

13.2 NUCLEO-F411RE UART GPIO PINS

There are three UART modules on the Nucleo-F411RE development board. The following are the GPIO pins for these modules:

UART Module	Signal	GPIO Pin
UART1	TX	PA_15, PB_6, PA_9
UART1	RX	PB_7, PB_3, PA_10
UART1	CTS	PA_11
UART2	TX	PA_2 (**PC communication**)
UART2	RX	PA_3 (**PC communication**)
UART2	CTS	PA_0
UART2	RTS	PA_1
UART6	RX	PC_7
UART6	TX	PC_6
UART6	RTS	PA_12

Notice that UART2 is used to communicate with the PC and is not freely available as a general purpose serial communication module.

13.3 PROJECT 1—TWO NUCLEO BOARDS COMMUNICATING THROUGH UART

13.3.1 Description

In this project two Nucleo-F411RE development boards called **Sender Node** and **Receiver Node** are connected through UART interface. A temperature sensor is connected to the **Sender Node**. This board reads the ambient temperature and sends it to the **Receiver Node** every second where it is displayed on the PC screen.

13.3.2 Aim

The aim of this project is to show how the UART module can be used on the Nucleo-F411RE development board and also how it can be programmed using Mbed.

13.3.3 Block Diagram

The block diagram of the project is shown in Fig. 13.3 where two Nucleo-F411RE development boards are connected to each other. The temperature sensor is connected to the **Sender Node** and temperature readings are displayed on the PC screen connected to the **Receiver Node**.

13.3.4 Circuit Diagram

Fig. 13.4 shows the circuit diagram of the project. SDA and SCL pins of the TMP102 temperature sensor is connected to pins PB_7 and PB_6 of the **Sender Node**, respectively. UART6 TX pin (PC_6) of the **Sender Node** is connected to UART6 RX pin (PC_7) of the **Receiver Node**.

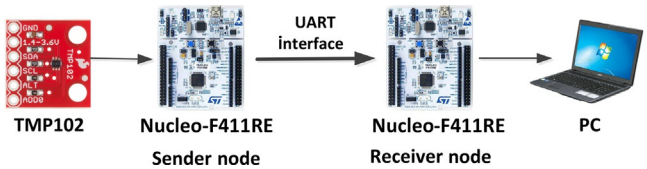

FIG. 13.3 Block diagram of the project.

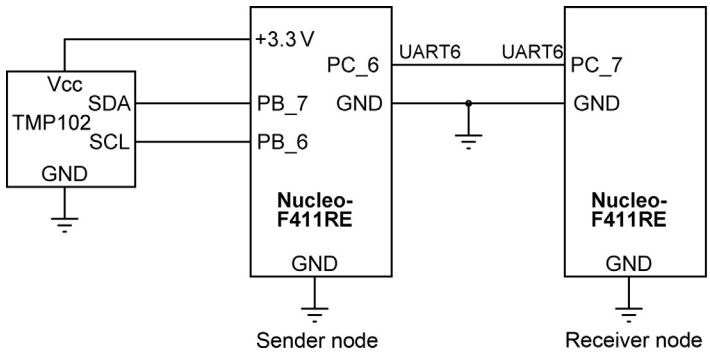

FIG. 13.4 Circuit diagram of the project.

```
SERVER NODE

BEGIN
    Configure UART6 and assign TX, RX to variable Sender
    Configure TMP102 module
    Set UART6 baud rate to 9600
    DO FOREVER
        Read the ambient temperature
        Convert into Degrees Centigrade
        Wait 1 second
    ENDDO
END

RECEIVER NODE

BEGIN
    Configure UART6 and assign TX, RX to variable Receiver
    Set UART6 baud rate to 9600
    DO FOREVER
        IF UART data is available THEN
            Read the data
            Display the data on PC screen
        ENDIF
    ENDDO
END
```

FIG. 13.5 Program PDL.

13.3.5 The PDL

Fig. 13.5 shows the program PDL.

13.3.6 Program Listing

Fig. 13.6 shows the **Sender Node** program listing (program: **Sender**). This program uses the I^2C compatible TMP102 temperature sensor module as was discussed in Chapter 11. At the beginning of the program variable **Sender** is assigned to UART6 pins PC_6 (TX pin) and PC_7 (RX pin). The program reads the ambient temperature every second and sends it to the **Receiver Node** over the UART interface using function **Sender.printf**. In this project, UART6 of the Nucleo board is used to send the temperature data. The baud rate of the communication is set to 9600.

The **Receiver Node** program listing (program: **Receiver**) is shown in Fig. 13.7. At the beginning of the program variable **Receiver** is assigned to UART6 pins PC_6 and PC_7.

This program reads the temperature data using UART6 again at 9600 baud. Function **receiver.scanf** is used to read the data from the UART. The received data is displayed on the PC monitor connected to the **Receiver Node** using function **pc.printf**.

A typical display from the Receiver Node is shown in Fig. 13.8.

```
/*********************************************************************

                            UART - SENDER NODE
                            ==================

This is an I2C based temperature project. In this project a TMP102 type
temperature sensor is connected to a Nucleo-F411RE development board (
called the Sender Node). The program reads teh ambient temperature and
then sends it to the Receiver Node through the UART6 module of the
development board. The baud rate is set to 9600.

Author: Dogan Ibrahim
Date  : September 2018
File  : Sender
*********************************************************************/
#include "mbed.h"
I2C TMP102(PB_7, PB_6);                         // I2C SDA, SCL
Serial Sender(PC_6, PC_7);                      // UART6 TX, RX

const int TMP102Address = 0x90;                 // TMP102 address
char ConfigRegister[3];                         // Config register
char TemperatureRegister[2];                    // Temperature Register
float Temperature;

//
// This function configures the TMP102 after power up or reset
//
void ConfigureTMP102()
{
    ConfigRegister[0] = 0x01;                   // Point to Config Register
    ConfigRegister[1] = 0x60;                   // Upper byte
    ConfigRegister[2] = 0xA0;                   // Lower byte
    TMP102.write(TMP102Address, ConfigRegister, 3); // Write 3 bytes
}


int main()
{
    unsigned short M;
    char L;

    ConfigureTMP102();                              // Configure TMP102
    ConfigRegister[0] = 0x00;                       // Point to Temp Register
    TMP102.write(TMP102Address, ConfigRegister, 1); // Write 1 byte
    Sender.baud(9600);                              // Sender baud rate
```

FIG. 13.6 Sender Node program listing.

(Continued)

```
    //
    // Read the ambient temperature, convert into degrees Centigrade and then
    // send it to the Receiver Node over the UART
    //
        while(1)
        {
            TMP102.read(TMP102Address, TemperatureRegister, 2);
            M = TemperatureRegister[0] << 4;
            L = TemperatureRegister[1] >> 4;
            M = M + L;
            Temperature = 0.0625 * M;
            Sender.printf("%f\n\r", Temperature);
            wait(5.0);
        }
    }
```

FIG. 13.6, CONT'D

```
/**************************************************************************

                            UART - RECEIVER NODE
                            ====================

In this project the Nucleo-F411RE development board (called the Receiver
Node)receives the ambient temperature readings from teh Sender Node and
then displays them on the PC screen. UART6 module of the development
board is used withe the baud rate set to 9600.

Author: Dogan Ibrahim
Date  : September 2018
File  : Receiver
**************************************************************************/
#include "mbed.h"

Serial pc(USBTX, USBRX);                            // PC interface
Serial Receiver(PC_6, PC_7);                        // UART6 TX, RX


int main()
{
    float Temperature;
    Receiver.baud(9600);                            // Sender baud rate

//
// Read the ambient temperature from the Sender Node and then display
// on the PC screen. First check to make sure that data is available
// before attempting to read from UART
//
    while(1)
    {
        if(Receiver.readable())
        {
            Receiver.scanf("%f", &Temperature);
            pc.printf("\n\rTemperature = %5.2f", Temperature);
        }
    }
}
```

FIG. 13.7 Receiver Node program listing.

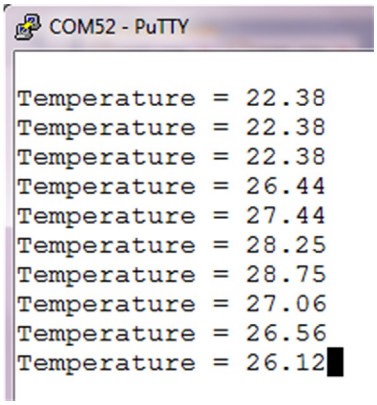

FIG. 13.8 Typical display from the Receiver Node.

13.4 SUMMARY

In this chapter we have learned about the following:

- UART
- Nucleo-F411RE development board UART modules
- Mbed UART functions
- A project where two Nucleo-F411RE development boards are connected together through one of their UART ports and they communicate with each other.

13.5 EXERCISES

1. What does the acronym UART stand for?
2. Explain the minimum signals that can be used when two devices want to communicate using UARTs.
3. What is the baud rate? What are the typical baud rates?
4. Two Nucleo-F411RE development boards are to be connected via the UART interface. Write a program to show how the data sent by one device can be received and displayed on the PC screen by the other device.
5. What are USBTX and USBRX?
6. Explain how sensor data can be received by a Nucleo-F411RE development board and then displayed on the PC screen. Give an example project with the code.